

# Canteen Management System

DBMS Course Project Report



**Student Name:** Hansith Raj Kasani

**Roll Number:** 25WU0102117

**Course:** Database Management Systems (DBMS)

**Academic Year:** 2025–2029

**Project Type:** DBMS Course Project

**Branch:** CSE - AIML

## 2. Abstract

The Canteen Management System is a database-driven web application designed to manage menu items, customers, orders, order items, ingredients, recipes, kitchen operations, stock usage, billing, and payments in a structured relational database. The project uses MySQL as the database layer, Python 3 with Flask for backend services, Jinja2 templates and vanilla JavaScript for the user interface, and mysql-connector-python for database connectivity. The system emphasizes normalized data, referential integrity, transaction management, validation, controlled error handling, and reporting. The order-placement workflow is transactional so that Orders, OrderItem, KitchenTask, and StockUsage changes are committed together or rolled back together. Required reports include daily sales, popular items, category sales, pending orders, ingredient consumption, cancellations, and revenue.

## 3. Introduction and Problem Statement

The canteen generates related operational data continuously. Customers place orders, orders contain multiple food items, food items require ingredients, kitchen staff process tasks, ingredients are consumed, bills are generated, and payments settle bills.

The application models this complete workflow using a relational database rather than storing all information in a single table. Separate entities reduce duplication and allow relationships to be enforced through primary keys, foreign keys, constraints, and transactions.

### 3.1 Problem Statement

A canteen operator needs a reliable way to maintain menu information, accept customer orders, track kitchen preparation, manage ingredient stock, generate bills, record payments, and produce management reports.

Without a centralized relational system, repeated data, incorrect stock values, unavailable-item ordering, invalid status changes, and inconsistent payment information can occur. The proposed system addresses these issues through normalized tables, validation rules, triggers, and transactional application logic.

## 4. Objectives and Scope

### 4.1 Objectives

- Maintain menu categories, food items, recipes, ingredients, customers, kitchen staff, orders, order items, kitchen tasks, stock usage, bills, and payments in MySQL.
- Provide CRUD operations for menu maintenance and operational records through a Flask web interface.
- Support cart-based order entry with live totals and both client-side and server-side validation.
- Ensure the complete order-placement transaction is atomic across Orders, OrderItem, KitchenTask, and StockUsage.
- Enforce kitchen status transitions and staff assignment.
- Identify low-stock ingredients and support manual stock issue/restock operations.
- Generate bills from served orders and validate payment amounts.

- Provide the seven required reports using joins, subqueries, aggregation, and views.

## 4.2 Scope

The project covers database-backed canteen operations and a web interface. It includes menu maintenance, order placement, kitchen workflow, inventory/ingredient management, billing, payments, and reports.

The scope does not include physical point-of-sale hardware, biometric systems, external payment gateways, or IoT hardware. The application is intended to run locally using the specified Python/Flask/MySQL stack.

## 5. Software and Hardware Requirements

Category | Requirement

Operating System | Windows / macOS / Linux

Backend | Python 3 + Flask

Database | MySQL

DB Connectivity | mysql-connector-python

Templates | Jinja2

Frontend | HTML5, CSS, vanilla JavaScript

Build process | No build step

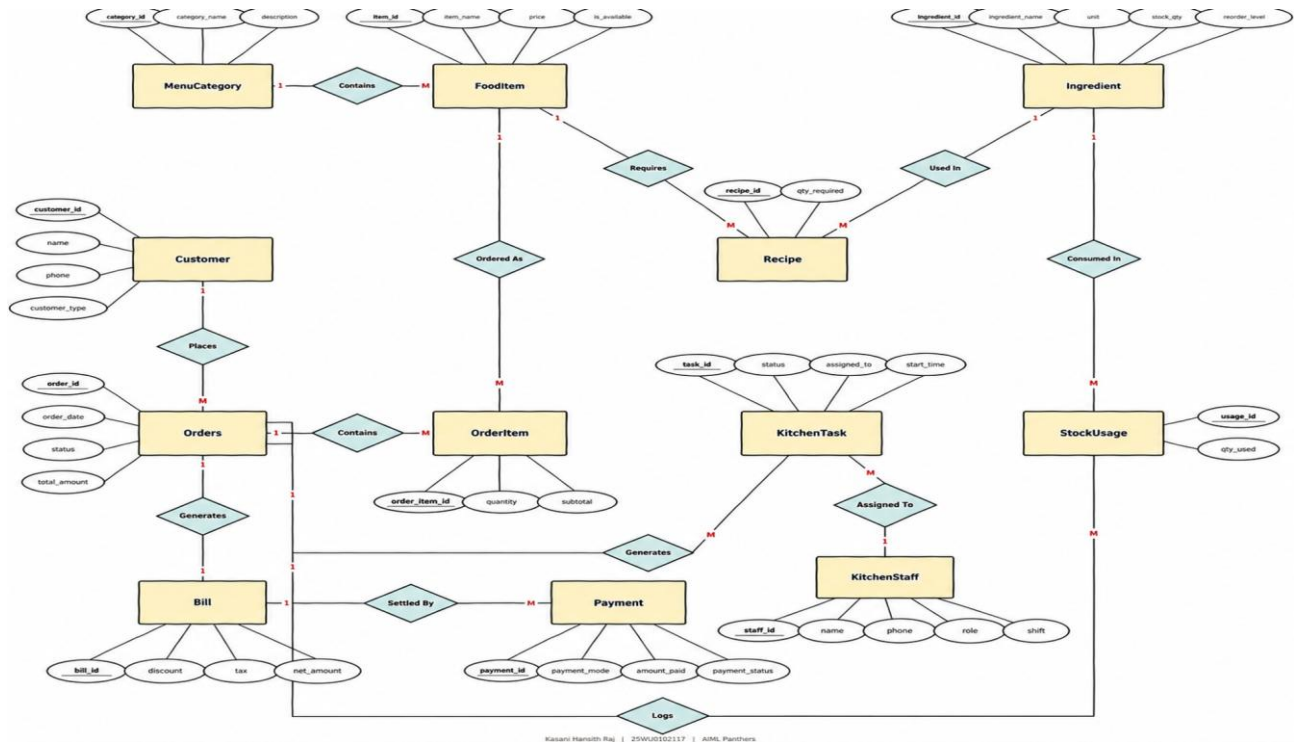
Testing | pytest

Browser | Modern Chromium / Firefox / Safari

Recommended Hardware | Dual-core processor, 8 GB RAM, 10 GB free storage

## 6. ER Diagram

The ER diagram models the twelve core entities and their relationships.



## 7. Relational Schema and Normalization

### 7.1 Relational Schema

MenuCategory | category\_id | — | category\_name, description

FoodItem | item\_id | category\_id → MenuCategory | item\_name, price, is\_available

Customer | customer\_id | — | name, phone, email, customer\_type

Ingredient | ingredient\_id | — | ingredient\_name, unit, stock\_quantity, reorder\_level, cost\_per\_unit

Recipe | recipe\_id | item\_id → FoodItem; ingredient\_id → Ingredient | qty\_required

KitchenStaff | staff\_id | — | name, phone, role, shift

Orders | order\_id | customer\_id → Customer | order\_date, status, total\_amount

OrderItem | order\_item\_id | order\_id → Orders; item\_id → FoodItem | quantity, unit\_price, subtotal

KitchenTask | task\_id | order\_id → Orders; staff\_id → KitchenStaff | status, start\_time, end\_time

StockUsage | usage\_id | ingredient\_id → Ingredient; order\_id → Orders | qty\_used, usage\_date

Bill | bill\_id | order\_id → Orders | bill\_date, discount, tax, net\_amount

Payment | payment\_id | bill\_id → Bill | payment\_mode, amount\_paid, payment\_date, payment\_status

## 7.2 Normalization

The schema follows 1NF, 2NF, and 3NF principles by keeping attributes atomic, avoiding repeating groups, and separating entity-specific non-key attributes. Recipe resolves the FoodItem–Ingredient many-to-many relationship and OrderItem represents individual order lines.

## 8. Data Dictionary

MenuCategory | category\_id | INT | PK; category identifier

MenuCategory | category\_name | VARCHAR(100) | Category name

MenuCategory | description | VARCHAR(255) | Category description

FoodItem | item\_id | INT | PK; food item identifier

FoodItem | category\_id | INT | FK to MenuCategory

FoodItem | item\_name | VARCHAR(100) | Food item name

FoodItem | price | DECIMAL(10,2) | Selling price; non-negative

FoodItem | is\_available | BOOLEAN | Availability flag

Customer | customer\_id | INT | PK

Customer | name | VARCHAR(100) | Customer name

Customer | phone | VARCHAR(15) | Unique contact number

Customer | email | VARCHAR(100) | Email address

Customer | customer\_type | VARCHAR(30) | Customer category/type

Ingredient | ingredient\_id | INT | PK

Ingredient | ingredient\_name | VARCHAR(100) | Ingredient name

Ingredient | unit | VARCHAR(30) | kg/litre/etc.

Ingredient | stock\_quantity | DECIMAL(10,2) | Current stock

Ingredient | reorder\_level | DECIMAL(10,2) | Low-stock threshold

Ingredient | cost\_per\_unit | DECIMAL(10,2) | Cost per kg/litre/etc.

Recipe | recipe\_id | INT | PK

Recipe | item\_id | INT | FK to FoodItem

Recipe | ingredient\_id | INT | FK to Ingredient

Recipe | qty\_required | DECIMAL(10,3) | Ingredient quantity required per item

KitchenStaff | staff\_id | INT | PK

KitchenStaff | name | VARCHAR(100) | Staff name

KitchenStaff | phone | VARCHAR(15) | Unique phone number

KitchenStaff | role | VARCHAR(50) | Kitchen role

KitchenStaff | shift | VARCHAR(30) | Assigned shift

Orders | order\_id | INT | PK

Orders | customer\_id | INT | FK to Customer

Orders | order\_date | DATETIME | Order timestamp

Orders | status | VARCHAR(30) | Placed/Preparing/Served/Cancelled

Orders | total\_amount | DECIMAL(10,2) | Order total

OrderItem | order\_item\_id | INT | PK

OrderItem | order\_id | INT | FK to Orders

OrderItem | item\_id | INT | FK to FoodItem

OrderItem | quantity | INT | Positive ordered quantity

OrderItem | unit\_price | DECIMAL(10,2) | Price at order time

OrderItem | subtotal | DECIMAL(10,2) | Line subtotal

KitchenTask | task\_id | INT | PK

KitchenTask | order\_id | INT | FK to Orders

KitchenTask | staff\_id | INT | FK to KitchenStaff

KitchenTask | status | VARCHAR(30) | Kitchen task status

KitchenTask | start\_time | DATETIME | Task start

KitchenTask | end\_time | DATETIME | Task completion

StockUsage | usage\_id | INT | PK

StockUsage | ingredient\_id | INT | FK to Ingredient

StockUsage | order\_id | INT | FK to Orders

StockUsage | qty\_used | DECIMAL(10,3) | Quantity consumed

StockUsage | usage\_date | DATE | Usage date

Bill | bill\_id | INT | PK

Bill | order\_id | INT | FK to Orders

Bill | bill\_date | DATETIME | Billing timestamp

Bill | discount | DECIMAL(10,2) | Discount amount

Bill | tax | DECIMAL(10,2) | Tax amount

Bill | net\_amount | DECIMAL(10,2) | Final bill amount

Payment | payment\_id | INT | PK

Payment | bill\_id | INT | FK to Bill

Payment | payment\_mode | VARCHAR(30) | Cash/Card/UPI

Payment | amount\_paid | DECIMAL(10,2) | Paid amount

Payment | payment\_date | DATETIME | Payment timestamp

Payment | payment\_status | VARCHAR(30) | Payment state

## 9. SQL Commands Used

### 9.1 DDL

```
CREATE DATABASE IF NOT EXISTS CanteenManagement;  
USE CanteenManagement;
```

```
CREATE TABLE MenuCategory (  
    category_id INT PRIMARY KEY AUTO_INCREMENT,  
    category_name VARCHAR(100) NOT NULL,  
    description VARCHAR(255)  
);
```

```
CREATE TABLE FoodItem (  
    item_id INT PRIMARY KEY AUTO_INCREMENT,  
    category_id INT NOT NULL,  
    item_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    is_available BOOLEAN DEFAULT TRUE,  
    FOREIGN KEY (category_id) REFERENCES MenuCategory(category_id),  
    CHECK (price >= 0)  
);
```

```
-- The remaining project tables follow the same reviewed schema:  
-- Customer, Ingredient, Recipe, KitchenStaff, Orders, OrderItem,  
-- KitchenTask, StockUsage, Bill, Payment.
```

### 9.2 Representative SELECT

```
SELECT m.category_name, f.item_name, f.price  
FROM MenuCategory m  
JOIN FoodItem f  
ON m.category_id = f.category_id;
```

## 10. Queries with Outputs / Required Reports

### 10.2 Category sales — JOIN + aggregation

```
SELECT m.category_name,  
       SUM(oi.subtotal) AS total_amount  
FROM MenuCategory m  
JOIN FoodItem f ON m.category_id = f.category_id  
JOIN OrderItem oi ON f.item_id = oi.item_id  
JOIN Orders o ON oi.order_id = o.order_id  
WHERE o.status <> 'Cancelled'  
GROUP BY m.category_id, m.category_name;
```

### 10.3 Customer order count — JOIN + COUNT + GROUP BY

```
SELECT c.name AS customer_name,  
       COUNT(o.order_id) AS total_orders  
FROM Customer c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.name;
```

### 10.4 Above-average orders — subquery

```
SELECT *  
FROM Orders  
WHERE total_amount > (  
    SELECT AVG(total_amount)  
    FROM Orders  
);
```

### 10.5 Daily sales — aggregation

```
SELECT DATE(order_date) AS sales_date,  
       SUM(total_amount) AS daily_sales  
FROM Orders  
WHERE status <> 'Cancelled'  
GROUP BY DATE(order_date);
```

### 10.6 Popular item — JOIN + aggregation + ORDER BY

```
SELECT f.item_name,  
       SUM(oi.quantity) AS quantity_sold  
FROM OrderItem oi  
JOIN FoodItem f ON oi.item_id = f.item_id  
JOIN Orders o ON oi.order_id = o.order_id  
WHERE o.status <> 'Cancelled'  
GROUP BY f.item_id, f.item_name  
ORDER BY quantity_sold DESC  
LIMIT 1;
```

### 10.7 Pending orders — JOIN + filtering

```
SELECT o.order_id, c.name AS customer_name,  
       o.order_date, o.status, o.total_amount  
FROM Orders o  
JOIN Customer c ON o.customer_id = c.customer_id  
WHERE o.status = 'Placed'  
ORDER BY o.order_date;
```

### 10.8 Ingredient consumption — JOIN + aggregation

```
SELECT i.ingredient_name,  
       SUM(s.qty_used) AS total_consumed  
FROM StockUsage s  
JOIN Ingredient i ON s.ingredient_id = i.ingredient_id  
GROUP BY i.ingredient_id, i.ingredient_name;
```

## 10.9 Cancellations — JOIN + filtering

```
SELECT o.order_id, c.name AS customer_name,  
       o.total_amount  
FROM Orders o  
JOIN Customer c ON o.customer_id = c.customer_id  
WHERE o.status = 'Cancelled';
```

## 10.10 Revenue — aggregation

```
SELECT SUM(amount_paid) AS total_revenue  
FROM Payment  
WHERE payment_status = 'Paid';
```

## 11. Presentation-II Query: Net Profit of the Total Canteen

The solution is presented as three steps: total revenue, total ingredient cost, and net profit.

```
SELECT SUM(oi.subtotal) AS total_revenue  
FROM OrderItem oi  
JOIN Orders o ON oi.order_id = o.order_id  
WHERE o.status <> 'Cancelled';  
  
SELECT SUM(  
    oi.quantity * r.qty_required * i.cost_per_unit  
    ) AS total_ingredient_cost  
FROM OrderItem oi  
JOIN Orders o ON oi.order_id = o.order_id  
JOIN Recipe r ON oi.item_id = r.item_id  
JOIN Ingredient i ON r.ingredient_id = i.ingredient_id  
WHERE o.status <> 'Cancelled';  
  
SELECT  
    SUM(oi.subtotal)  
    - SUM(oi.quantity * r.qty_required * i.cost_per_unit)  
    AS net_profit  
FROM OrderItem oi  
JOIN Orders o ON oi.order_id = o.order_id  
JOIN Recipe r ON oi.item_id = r.item_id  
JOIN Ingredient i ON r.ingredient_id = i.ingredient_id  
WHERE o.status <> 'Cancelled';
```

## 12. Application Implementation Details

### 12.1 Technology Stack

Layer | Technology | Purpose

Backend | Python 3 + Flask | Routes, business logic, validation, transactions

Templates | Jinja2 | Server-rendered HTML

Frontend | HTML5 + CSS + vanilla JavaScript | Forms, cart UI, live totals, client validation

Database | MySQL | Persistent relational data

DB Driver | mysql-connector-python | MySQL connectivity

Testing | pytest | Automated tests against seeded data

Build | No build step | Direct Flask/Jinja/JS execution

## 12.2 Application Modules

- DB connection layer with a transaction() context manager.
- Menu maintenance: CRUD for MenuCategory, FoodItem, and Recipe with search/filter.
- Order entry: cart interface, live totals, client/server validation, and atomic order placement across Orders, OrderItem, KitchenTask, and StockUsage.
- Kitchen status board with enforced Pending → Cooking → Ready → Served transitions and staff assignment.
- Ingredient stock page with low-stock flagging and manual issue/restock.
- Billing and payment: generate bill from served order and enforce amount\_paid ≤ net\_amount.
- Reports page containing all required reports, with comments identifying the SQL technique used.

## 12.3 Transaction and Error-Handling Rules

Every multi-table write uses transaction() and is committed only after all related operations succeed. Failures trigger rollback. DB calls are protected by try/except, real errors are logged server-side, and users receive controlled plain-English messages. Every form uses client-side and server-side validation.

## 12.4 Project Structure

```
project/
├── app.py
├── db.py
├── requirements.txt
├── PROJECT_BRIEF.md
├── TEST_PLAN.md
├── db/schema.sql
├── db/sample_data.sql
├── templates/
├── static/
├── tests/
└── screenshots/
```

## 13. Testing

Test Case | Expected Result | Status

Database connection | Application connects to MySQL using db.py | To be verified

Menu CRUD | Insert/update/delete operations persist correctly | To be verified

Unavailable item validation | Unavailable food item cannot be ordered | To be verified

Order transaction | Orders, OrderItem, KitchenTask, StockUsage commit together | To be verified

Order rollback | Failure in any order step rolls back all related writes | To be verified

Kitchen transition | Only valid Pending → Cooking → Ready → Served transitions succeed | To be verified

Low stock | Ingredients at/below reorder level are flagged | To be verified

Billing | Bill is generated only for a served order | To be verified

Payment limit | amount\_paid cannot exceed net\_amount | To be verified

Reports | All seven required reports return correct relational results | To be verified

Error handling | Invalid operations show controlled plain-English errors | To be verified

Final browser walkthrough and screenshot capture should be performed after the application is operational.

## 14. Conclusion and Future Enhancements

### 14.1 Conclusion

The Canteen Management System provides a normalized relational solution for menu, order, kitchen, inventory, billing, payment, and reporting workflows. Relational constraints and transactions preserve consistency.

### 14.2 Future Enhancements

- Authentication and role-based access control.
- Real-time kitchen notifications.
- Online payment gateway integration.
- Automated stock deduction and reorder alerts.
- Advanced sales and ingredient analytics.
- Cloud deployment and automated backups.

## 15. References

- MySQL Documentation — SQL, constraints, triggers, and transactions.
- Python 3 and Flask documentation.
- mysql-connector-python documentation.
- Reference report supplied by the user: Smart Parking Lot Management System DBMS Course Project Report.

## 16. Appendix

Presentation-II formula: Total Revenue = non-cancelled order-item subtotals; Total Ingredient Cost = order quantity × recipe quantity required × ingredient cost per unit; Net Profit = Total Revenue – Total Ingredient Cost.